

Terrain & Grass Pack

Documentation v1.0



A versatile nature texture pack designed for terrain creation, environment prototyping and outdoor scenes. Perfect for developers looking for a simple and practical foundation for natural outdoor environments.

This package includes a collection of terrain textures, grass billboards and rock textures that can be used to quickly build natural environments for games, prototypes and stylized worlds.

Ideal for:

- Open world prototypes
- Survival games
- Stylized environments
- Indie projects
- Mobile games
- Nature scenes
- Rapid level blockout

Included Content

- [Ground and terrain textures](#)
 - [Add preconfigured Terrain Layers](#)
- [Grass billboard textures](#)
 - [Adding grass to a Terrain component](#)
- [Large and small 3D rocks](#)
- [Layered Skybox](#)
- [Basic Water shader](#)
- [Water Lily](#)
- [Day & Night Cycle System](#)

Features

- Lightweight and easy to use

- Suitable for fast prototyping
- Modular workflow friendly
- Reusable across multiple projects
- Optimized for performance
- Beginner friendly

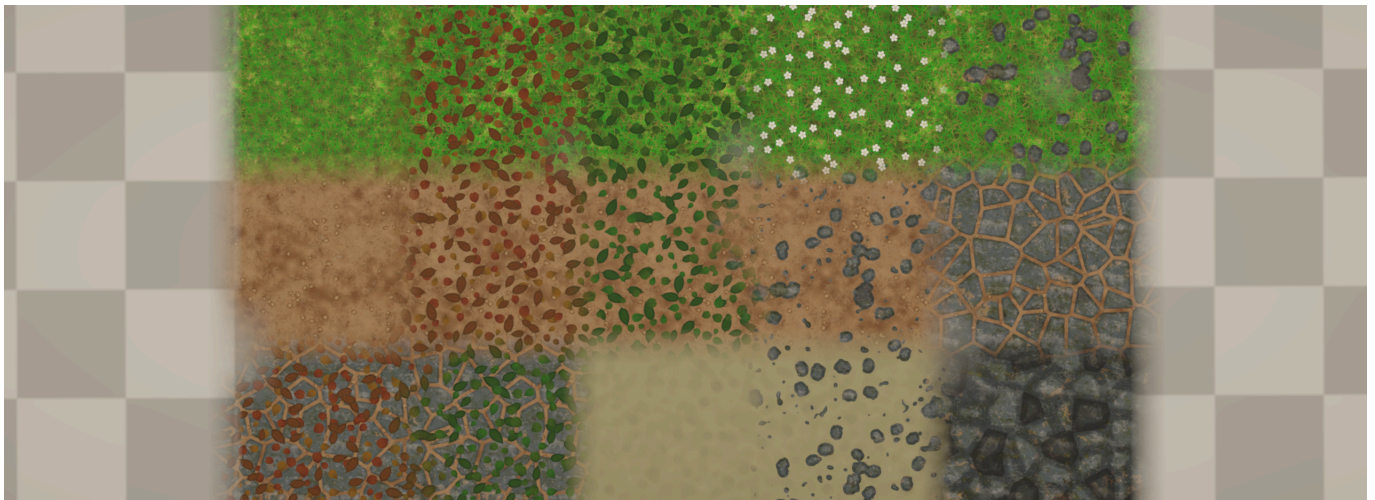
Compatible With

- URP by default. Built-in and HDRP support have not been fully tested, so additional material or shader updates may be required in those pipelines.

Ground and terrain textures

The folder `Assets/FlipGameDev/Terrain&GrassPack/Art/Textures/Terrain_Textures` contains 15 terrain textures with normal maps, including:

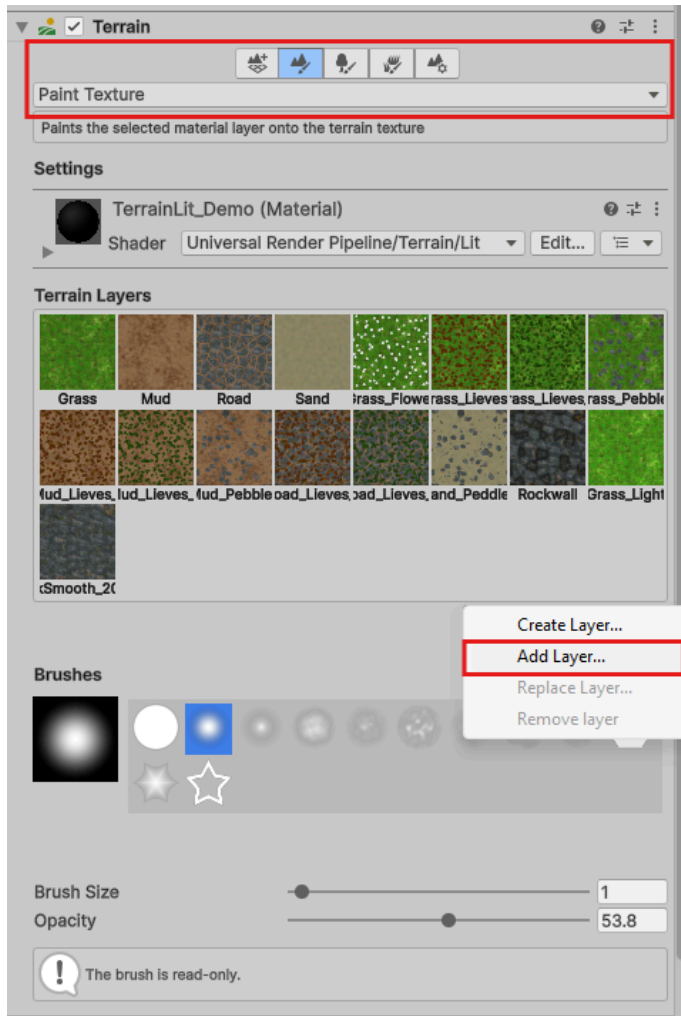
- 5 grass textures
- 4 soil textures
- 2 sand textures
- 3 pavement textures
- 1 rock texture



Add preconfigured Terrain Layers

The folder `Assets/FlipGameDev/Terrain&GrassPack/Terrain_Settings/Terrain_Layers` contains preconfigured Terrain Layers ready to use in Unity's Terrain component. To add them to a newly created terrain:

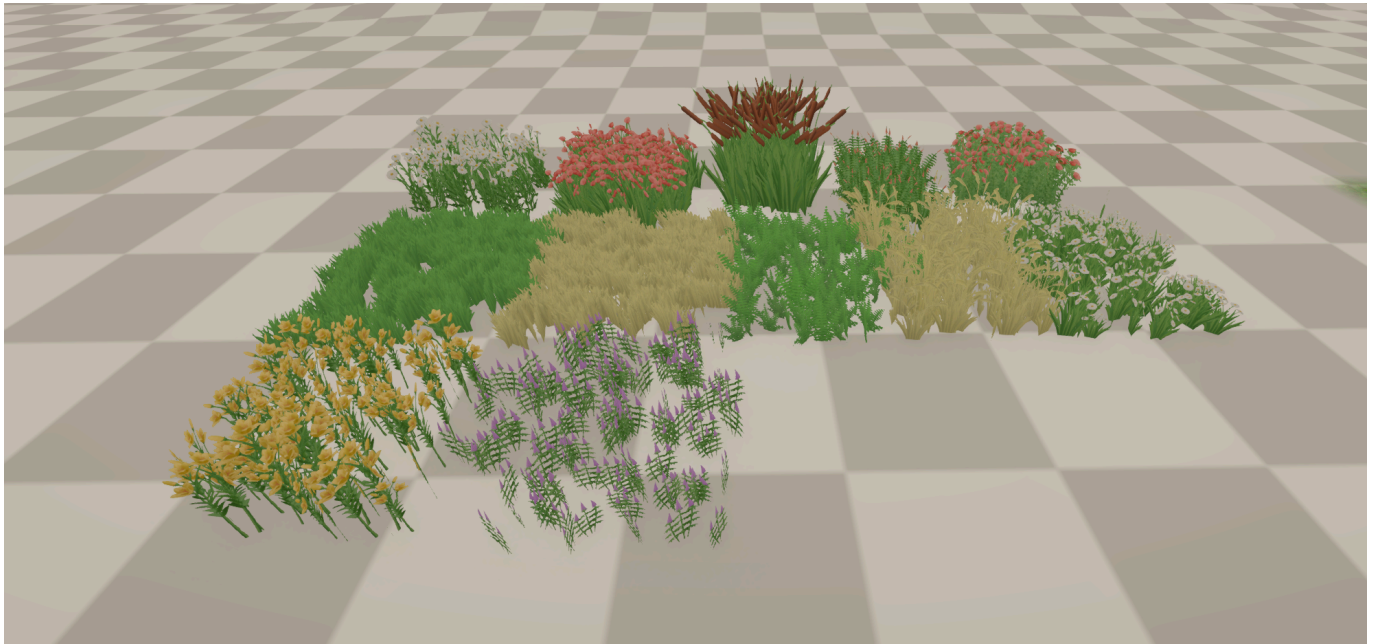
- Open the `Paint Terrain` tab.
- Select `Paint Texture` mode.
- Click the `Edit Terrain Layers...` button.
- Click `Add Layer....`
- Choose the terrain layers you want to add to your terrain.



Grass billboard textures

The folder `Assets/FlipGameDev/Terrain&GrassPack/Art/Textures/Grass_Billboard` contains 11 grass billboard textures, including:

- Braske Ferns 1 (grayscale)
- Braske Ferns 2
- Cattail
- Grass
- Orchid
- Pampas
- Rose 1
- Rose 2 (with grass around)
- White Daisies 1
- White Daisies 1 (with grass around)
- Wild Lupine

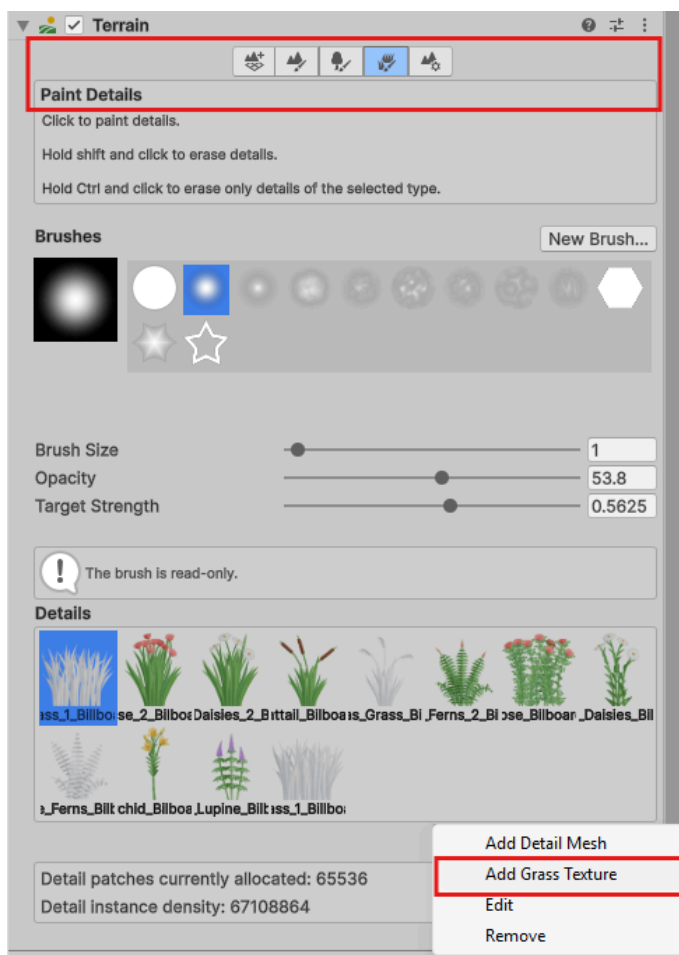


The folder `Assets/FlipGameDev/Terrain&GrassPack/Art/Textures/Grass_Billboard_Normal` contains the normal maps for these textures, in case you want to implement grass using a `Detail Mesh`.

Adding grass to a Terrain component

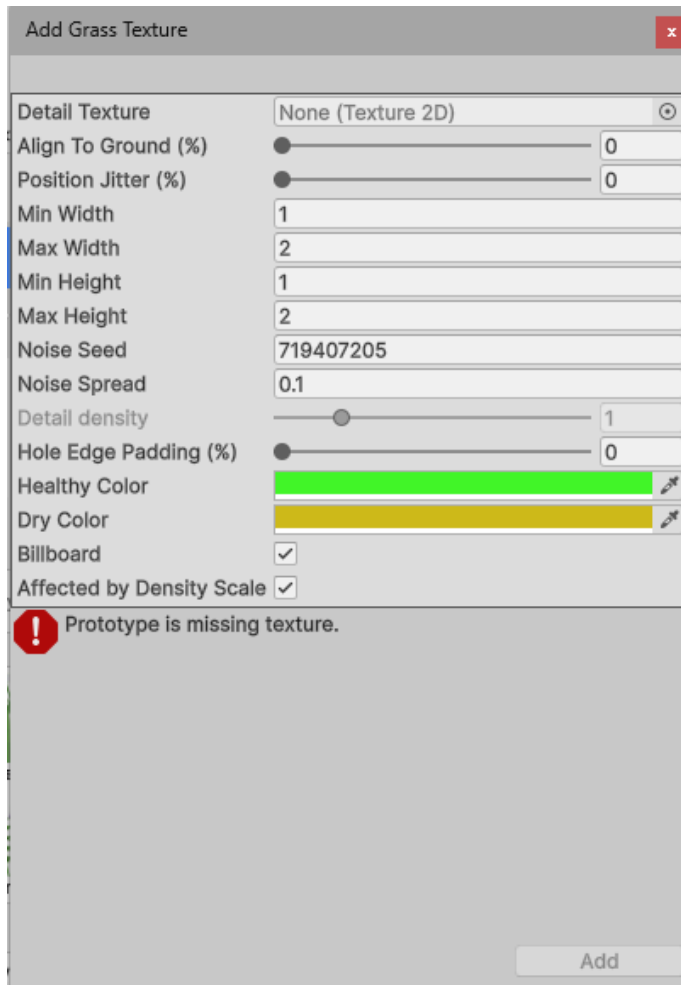
In the `Paint Details` tab:

- Click the  `Edit Details...` button.
- Select `Add Grass Texture`.



A window called **Add Grass Texture** will open.

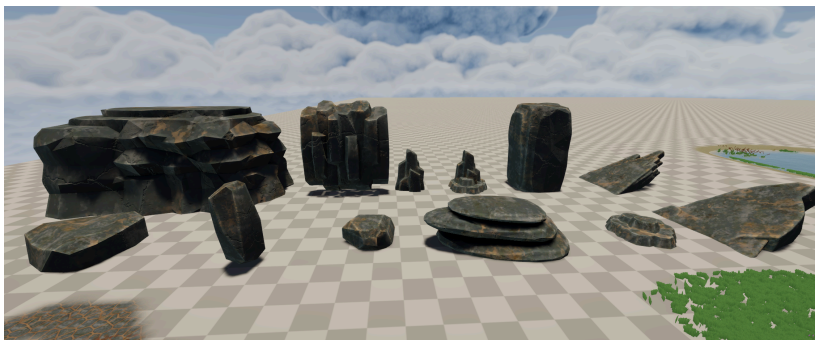
- Add the desired texture to the **Detail Texture** field.
- If the texture is grayscale, you can adjust its color using **Healthy Color** and **Dry Color**.
- Finally, click **Add**.



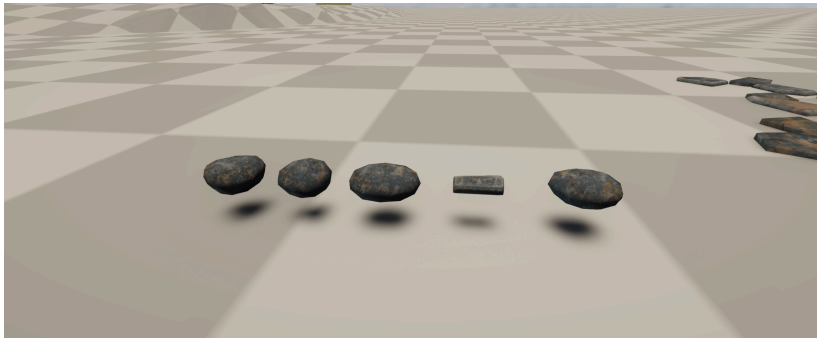
Large and small 3D rocks

The folder **Assets/FlipGameDev/Terrain&GrassPack/Prefabs** contains several rock prefabs already configured with colliders.

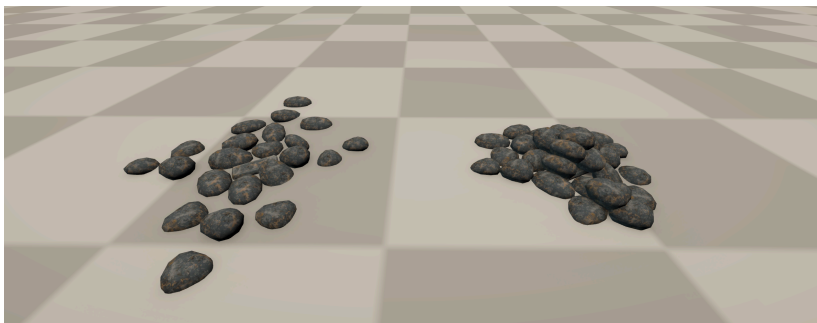
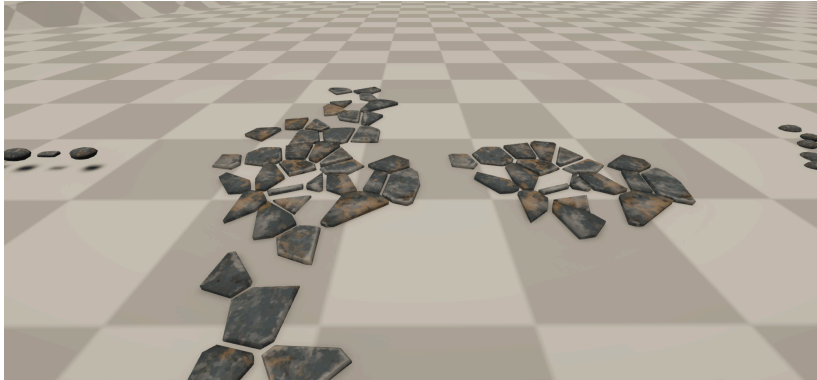
- In the **Rocks** folder you will find the larger rock models.



- In the **Rocks_Small** folder you will find the smaller rock models.



- In the **Rocks_Variation** folder you will find variations such as small stone piles and pavement outcroppings.

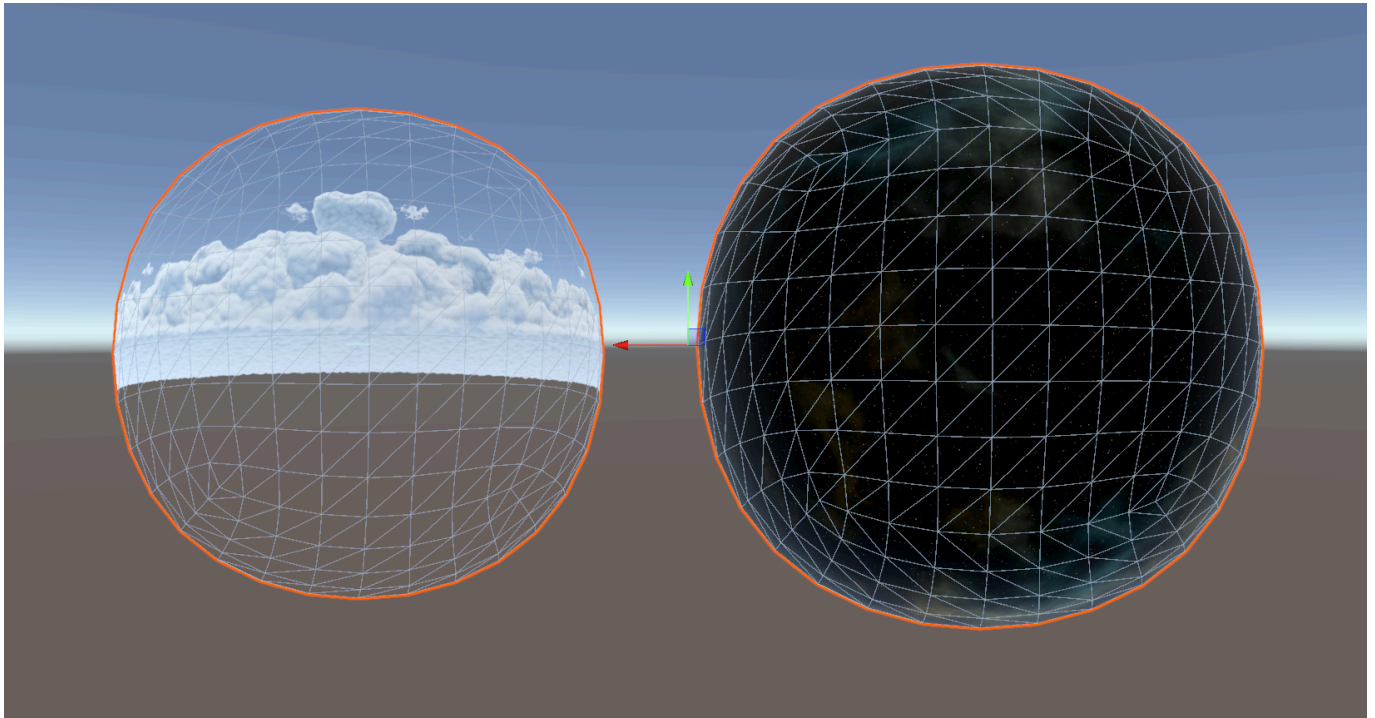


Layered Skybox

The **Layered Skybox** textures are designed to be used together with Unity's **Procedural Skybox**. The Unity Procedural Skybox already has solid built-in settings, so this integration is simple and delivers strong visual results.

This setup includes:

- A cloud texture in the Cloudscape style, with a focus on large, dramatic clouds.
- A night sky texture.
- Spherical meshes generated from a cube to reduce distortion and texture artifacts.



Both skybox layers are already configured in the prefab [Day-Night-Manager](#) under [Assets/FlipGameDev/Terrain&GrassPack/Prefabs](#). The [Day & Night Cycle System](#) section explains how this system manages the layered skybox.

Basic Water shader

This project includes a basic water shader created with [Shader Graph](#). The shader is located in [Assets/FlipGameDev/Terrain&GrassPack/Art/Shaders/Water](#), and its features are organized into [Sub Graphs](#). The water shader includes:

- Cat's Paw effects
- Depth fade
- Edge foam
- Refraction
- Wave motion

The material uses two textures located in [Assets/FlipGameDev/Terrain&GrassPack/Art/Textures/Water](#):

- Normal Map (a blend of Voronoi and a Substance cloud texture)
- Blend Texture (Substance cloud texture)

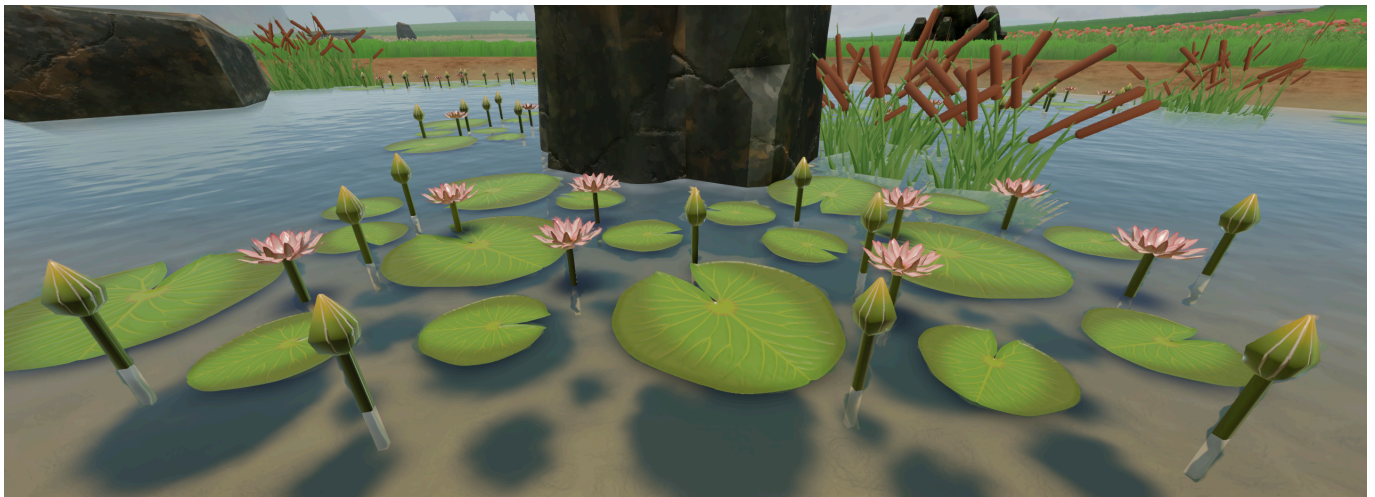
The fully configured material is in [Assets/FlipGameDev/Terrain&GrassPack/Art/Materials/Water](#).



Water Lily

Water lily assets were also added to enhance the water areas. In

[Assets/FlipGameDev/Terrain&GrassPack/Prefabs/WaterLily](#) you will find prefabs for individual elements (leaves and flowers) and grouped sets to make scene composition easier.



Day & Night Cycle System

1. Key Features

- **Modular Architecture:** Core brain, shared state, and independent modules.
- **Smart Time Scaling:** Custom Day/Night duration ratios (e.g., 2/3 Day, 1/3 Night).
- **Seamless Fades:** Automated fading for Sun/Moon light intensity, Stars alpha, and Moon emission.
- **Gradient Environments:** Fully supports Unity's Trilight Ambient Mode to change Sky, Equator, and Ground colors dynamically.

2. Core Architecture

The system is divided into three main layers to ensure high reusability and zero spaghetti code:

1. **DayNightState:** A shared data container holding the current time, angle, and day/night status.
2. **IDayNightModule:** A standard interface ([Initialize](#), [Tick](#), [Dispose](#)) that all logic scripts inherit from.

3. **DayNightBrain:** The central orchestrator. It sits on your GameObject, automatically detects any attached modules, and safely ticks them every frame.

3. Quick Start Guide

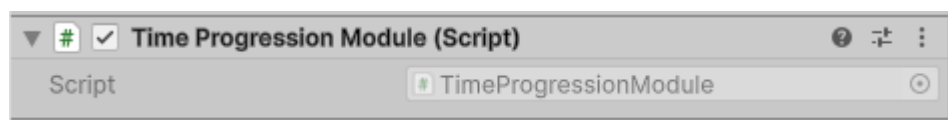
Follow these simple steps to add the cycle to your scene:

1. **Create the Manager:** Right-click in your Hierarchy and create an Empty GameObject. Name it **DayNightManager**.
2. **Add the Brain:** Attach the **DayNightBrain** component to it.
3. **Add Modules:** Attach the following components to the same GameObject (or its children):
 - **TimeProgressionModule**
 - **CelestialRotationModule**
 - **EnvironmentLightingModule**
 - **CelestialVisualsModule**
4. **Assign References:**
 - In the **Celestial Rotation Module**, assign your Sky (or Rotator) Transform.
 - In the **Environment Lighting** & **Celestial Visuals** modules, assign your Directional Lights (Sun, Moon) and renderers (Stars, Clouds, Moon).
5. **Tweak Settings:** Adjust the **Cycle Duration Minutes** in the Brain's State to 1, and tweak the Gradient colors to your game's art style. Play the scene!



4. Modules Breakdown

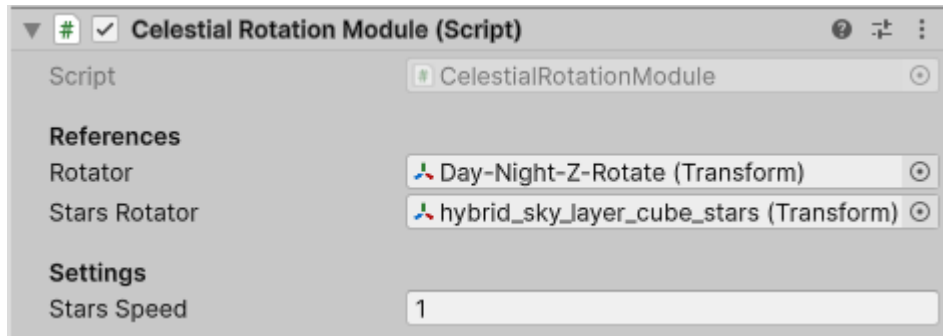
A. Time Progression Module



Responsible for pushing time forward. It calculates the speed of rotation based on the total cycle duration.

- **Logic:** By default, it splits the cycle into a 2/3 Day and 1/3 Night ratio, automatically adjusting the rotation speed based on the current angle.

B. Celestial Rotation Module



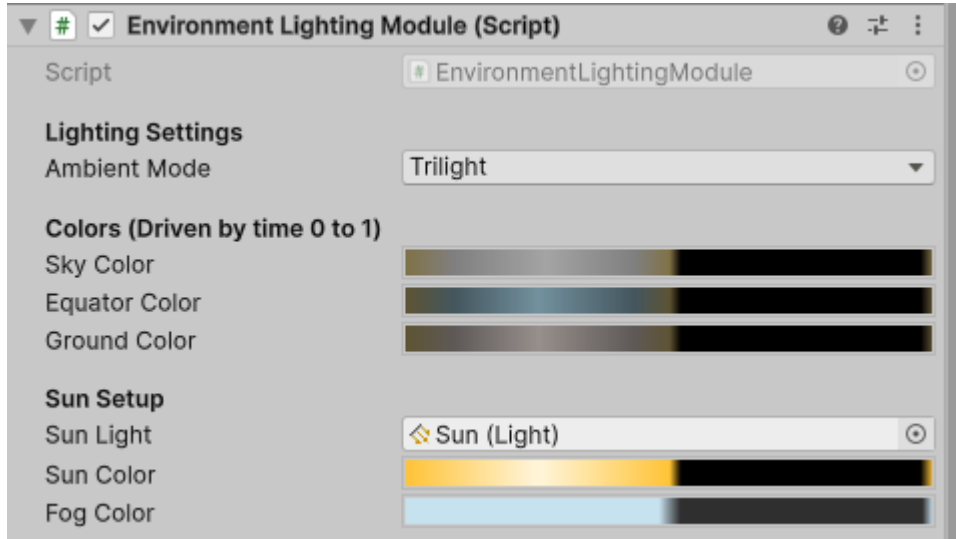
Handles the physical transformation of your skybox elements.

- **Rotator:** The main transform that spins 360 degrees (usually holding the Sun and Moon lights).
- **Stars Rotator / Speed:** An optional secondary transform for stars, allowing you to set a different rotation speed to create a parallax effect.

C. Environment Lighting Module

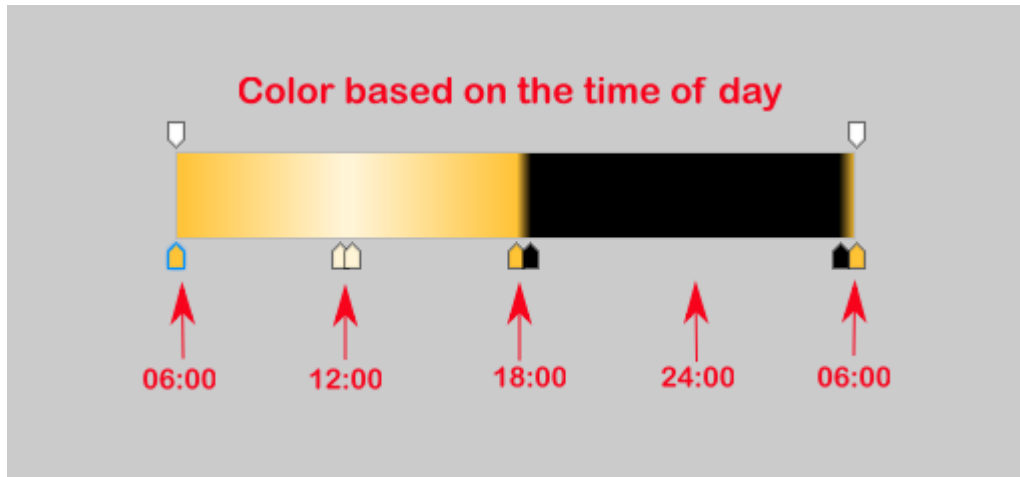
Controls the global ambient illumination of your scene. This module heavily relies on Unity's Gradient component to interpolate colors smoothly over time.

- **Ambient Mode:** Best used with *Trilight (Gradient)*.
- **Color Gradients:** Maps the time of day (0.0 to 1.0) to Sky, Equator, and Ground colors.
 - *Tip:* Ensure the 0% (far left) and 100% (far right) keys in your Gradients are exactly the same color to guarantee a seamless loop.



Understanding the Gradient Mapping

To configure your colors correctly, you need to understand how the system maps the "Time of Day" to the Gradient UI from left (0%) to right (100%).



The cycle is driven by degrees (0° to 360°), which perfectly translates to the 0% to 100% space in the gradient:

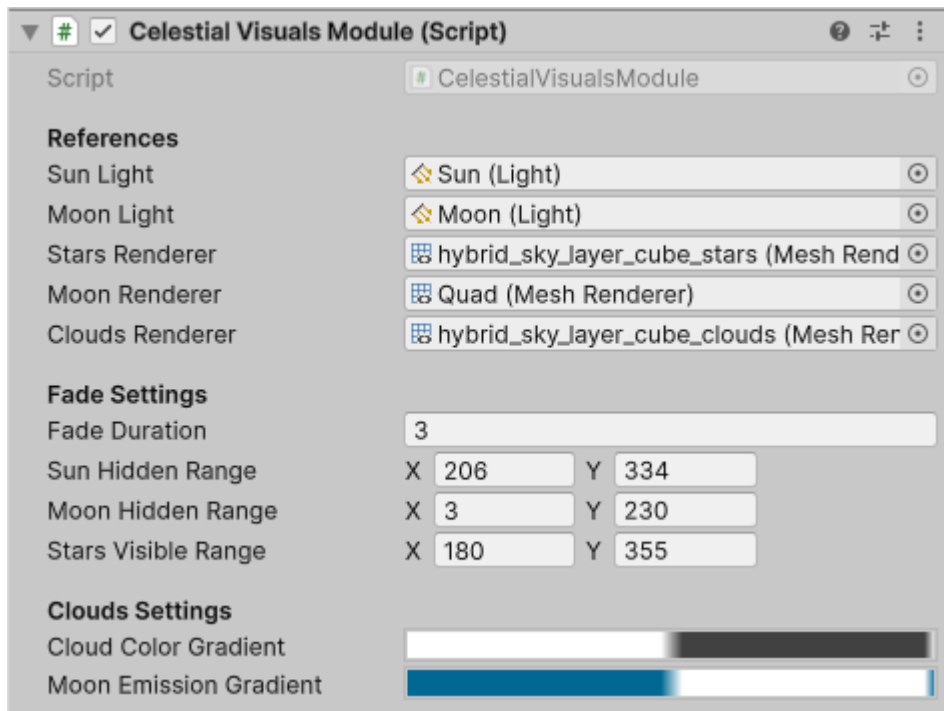
- **0% (Far Left):** 06:00 - Sunrise (Angle 0°)
- **25% (Mid-Left):** 12:00 - Midday (Angle 90°)
- **50% (Center):** 18:00 - Sunset (Angle 180°)
- **75% (Mid-Right):** 24:00 - Midnight (Angle 270°)
- **100% (Far Right):** 06:00 - Sunrise of the next day (Angle 360°)

Configuration Steps & Best Practices:

1. **The Golden Rule for Seamless Loops:** The color key at the absolute **0%** mark must be exactly the same as the color key at the **100%** mark. If they differ, your sky will "pop" and change color abruptly when the cycle restarts.
2. **Day vs. Night Duration:** Notice that the day (0% to 50%) and night (50% to 100%) take up the exact same amount of space in the gradient interface. **Don't worry about the time duration here!** Even though the *Time Progression Module* makes the day last longer in real-time (e.g., $2/3$ of the cycle), the gradient only cares about the sun's angle. Just paint the left half as "Daytime" and the right half as "Nighttime".
3. **Trilight Setup:** For the best results, set the *Ambient Mode* to **Trilight**. This allows you to set three separate gradients (Sky, Equator, and Ground), giving you massive artistic control over how the horizon blends with the sky at sunset and midday.

D. Celestial Visuals Module

The visual powerhouse. It manages fades, emissive materials, and light intensities.



- **Hidden/Visible Ranges:** Vector2 fields that let you define exactly at which angles the Sun, Moon, or Stars should fade in or out (e.g., Sun Hidden Range: X: 206, Y: 334).
- **Memory Management:** Automatically clones materials at runtime (to protect your project assets) and safely cleans them up via the `Dispose()` method when destroyed, preventing Unity memory leaks.

5. Extending the System (For Programmers)

One of the biggest strengths of this asset is its extensibility. If you want to trigger gameplay events (like spawning monsters at night, or playing a rooster sound at sunrise), **do not** modify the existing scripts. Instead, create a new module!

Here is an example of how easy it is to create your own custom module:

```
using UnityEngine;
using GameSystem.DayNight;

public class SpawnerModule : MonoBehaviour, IDayNightModule
{
    private bool _wasDaytime;

    public void Initialize(DayNightState state)
    {
        _wasDaytime = state.IsDaytime;
    }

    public void Tick(DayNightState state, float deltaTime)
    {
        // Detect the exact moment day turns into night
        if (_wasDaytime && !state.IsDaytime)
        {
            SpawnNightMonsters();
        }
    }
}
```



```
    }

    _wasDaytime = state.IsDaytime;
}

public void Dispose()
{
    // Clean up any instantiated data here if necessary
}

private void SpawnNightMonsters()
{
    Debug.Log("The sun has set. Monsters are spawning!");
}
}
```

*Just attach this script to the **DayNightManager** GameObject, and the Brain will automatically find it and run it!*

6. Best Practices & Optimization

- **Performance:** The Brain caches all modules during **Awake()** using **GetComponentsInChildren**. There are zero **GetComponent** calls or allocations happening during **Update()**.
 - **Execution Order:** The **DayNightBrain** is tagged with **[DefaultExecutionOrder(-100)]**. This ensures the time and lighting are updated *before* your player scripts or cameras calculate their logic for the frame, preventing visual stuttering.
 - **Hierarchy:** You can place modules on child GameObjects to keep your manager visually organized. The Brain will still find them.
-